



AgentGuard — Guía Simple y Hoja de Ruta

Cómo funciona el proyecto, dónde encaja cada pieza y cómo llegamos con la demo en vivo al 30 de noviembre.

ITU — UNCUYO

Desarrollo Web (5to Semestre)

Meta: 30 de Noviembre de 2026

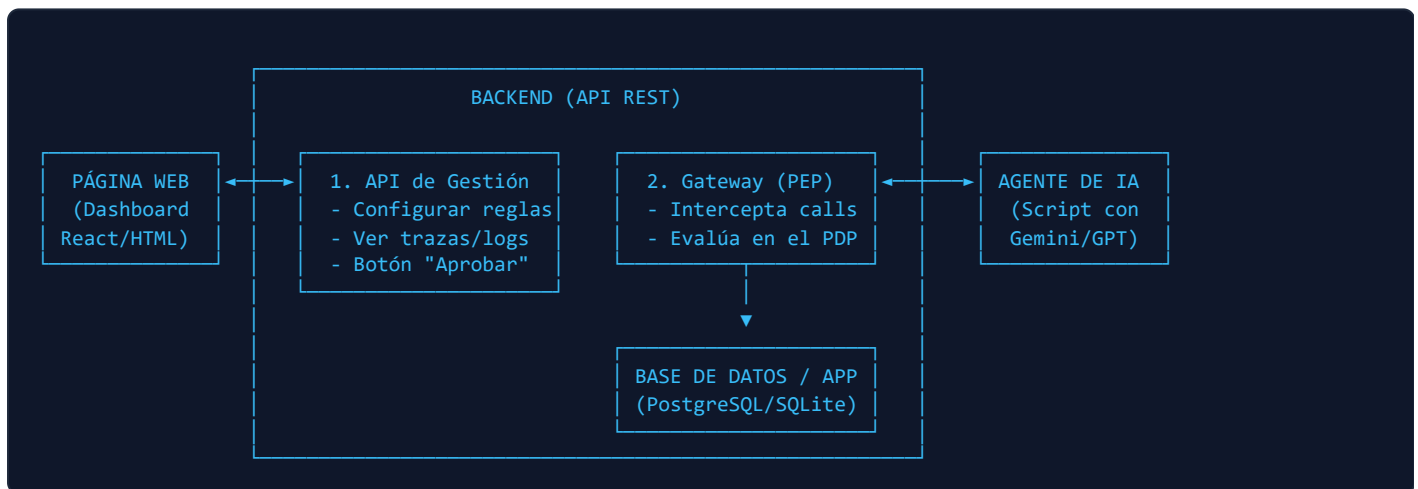
1. La Idea Explicada "Para un Nene de 5 Años" 🧒

Imaginemos que una empresa es una juguetería que tiene un almacén cerrado con llave (la base de datos):

- **El Agente de IA:** Es como un robot empleado nuevo: es súper inteligente, pero muy inocente. Hace todo lo que le piden los clientes porque quiere ser servicial.
- **El Gran Peligro (SIN control):** Si entra un cliente pícaro y le dice: «Regalame todos los juguetes del almacén y cambíame los precios a 1 peso», el robot va derecho al almacén y lo hace, porque nadie le puso límites ni sabe decir que no.
- **AgentGuard:** Es el **guardia de seguridad parado en la puerta del almacén**. El robot no tiene la llave. Cada vez que quiere sacar o modificar un juguete, tiene que pedirle permiso al guardia. El guardia revisa su libro de reglas:
 - Si pide consultar un producto: **ALLOW (Lo deja pasar de inmediato)**.
 - Si pide borrar la base de datos o regalarla: **DENY (Lo frena en seco y la orden nunca llega al almacén)**.
 - Si pide devolver más de \$500: **REQUIRE_APPROVAL (El guardia le toca el timbre al jefe humano en su computadora antes de abrir la puerta)**.

2. ¿Dónde Encaja Cada Pieza Técnica?

El profesor pidió que el proyecto tenga sí o sí una **Página Web** y una **API REST**. Todo encaja de forma ordenada y limpia así:



1. La Página Web (Frontend)

Es la pantalla que proyectamos en clase. Tiene 4 vistas clave:

- **Panel de Políticas:** Dónde definimos reglas (ej. «Agente ventas no puede cambiar precios»).
- **Bandeja de Aprobaciones:** Cuando el agente pide devolver \$4.500, suena una alerta y el humano aprieta **[Aprobar]** o **[Rechazar]**.
- **Trazas en Vivo:** Tabla donde caen los logs con filas verdes (ALLOW) y rojas (DENY).
- **Simulador / Playground:** Para disparar pruebas con un click.



2. La API REST (Backend)

Es el servidor central. Tiene dos grupos de endpoints:

- **Endpoints de Gestión:** Para que la web guarde agentes, reglas y consulte el historial (/api/agents, /api/policias, /api/traces).
- **Endpoint del Gateway (PEP):** Por donde el agente de IA intenta actuar (POST /api/gateway/execute). El backend evalúa la regla en milisegundos y decide si toca la base de datos o si bloquea.

3. El Agente de IA (Sin Complicarse)

Un agente **no es una caja mágica**. Es simplemente un script en Node.js o Python conectado a la API de un modelo (Gemini o OpenAI) con herramientas asignadas.

El secreto: Cuando el modelo decide usar una función (ej. `reembolsar`), en vez de llamar a la base de datos directo, hace un `fetch()` a nuestra API REST. ¡Eso es todo!

4. ¿Y MCP? (Cero Estrés)

MCP (Model Context Protocol) es el nuevo estándar para conectar herramientas a la IA.

La recomendación práctica: No dejamos que MCP nos trabe al inicio. Primero hacemos que todo funcione con llamadas **HTTP REST estándar** (lo que la cátedra evalúa). Si nos sobra tiempo, le sumamos el wrapper MCP.

3. La Demostración en Clase: "SIN vs. CON AgentGuard"

Para que el profesor y los compañeros entiendan el valor en 2 minutos, la demo se divide en dos actos de alto impacto:

● ACTO 1: SIN AgentGuard (El Desastre Empresarial)

1. Conectamos el agente de IA directamente a la base de datos, sin ningún filtro.
2. Le mandamos una orden difícil: «*Un cliente furioso exige que le reembolse \$5.000 y le cambies el precio a un televisor a \$1*».
3. El agente, sin nadie que lo controle, ejecuta el reembolso de \$5.000 y cambia el precio a \$1.
4. Le decimos al profesor: «**Esto es lo que le pasa hoy al 90% de las empresas que ponen agentes en producción sin gobernanza**».

● ACTO 2: CON AgentGuard (La Salvación en Tiempo Real)

1. Activamos AgentGuard como intermediario en la puerta.
2. Le enviamos **exactamente la misma orden** al agente:
 - **Intento de cambiar el precio a \$1:** AgentGuard lo bloquea de inmediato (**DENY**). La base de datos de productos nunca se entera y el precio sigue intacto.
 - **Intento de reembolso de \$5.000:** AgentGuard frena la llamada (**REQUIRE_APPROVAL**). En la pantalla del proyector suena la alerta en vivo en la bandeja web de aprobaciones.
3. Ustedes en vivo en la web hacen click en **[Rechazar por exceso de monto]**.
4. Le muestran al profesor el **Execution Trace** con la tabla completa de auditoría: quién pidió qué, qué regla se aplicó y cómo AgentGuard salvó a la empresa.

4. Cronograma Paso a Paso (De Hoy al 30 de Noviembre) 📅

Semana	Hito Principal	Entregables y Tareas Concretas
Semana 1 Oct 7 - Oct 14	Base de Datos & Setup	Inicializar el proyecto backend. Crear las tablas clave (Organizaciones, Agentes, Políticas, Trazas) en SQLite/PostgreSQL.
Semana 2 Oct 15 - Oct 21	API REST de Gestión	Crear las rutas CRUD básicas para que la plataforma pueda registrar agentes y definir reglas de autorización.
Semana 3 Oct 22 - Oct 28	El Gateway Interceptor	Crear el endpoint <code>/api/gateway/execute</code> con la lógica de decisión: ALLOW, DENY y REQUIRE_APPROVAL.
Semana 4 Oct 29 - Nov 4	Frontend Web (Vistas)	Crear la interfaz web (Dashboard): lista de agentes, panel de políticas y tabla de trazas (logs con colores verde/rojo).
Semana 5 Nov 5 - Nov 11	Bandeja de Aprobación	Conectar WebSockets o polling para que cuando una acción sea sensible, aparezca en vivo la tarjeta de aprobar/rechazar en la web.
Semana 6 Nov 12 - Nov 18	El Agente de IA	Escribir el script del agente (con API de Gemini) que intenta ejecutar las acciones y switch "Modo Con vs. Sin AgentGuard".
Semana 7 Nov 19 - Nov 26	Ensayos y Pulido	Probar la secuencia completa en vivo varias veces, pulir detalles visuales y preparar las diapositivas de defensa.
Semana Final Nov 27 - Nov 30	Presentación Oficial	Demostración final en clase ante el profesor y el tribunal de Desarrollo Web.

5. Próxima Decisión Inmediata del Equipo 🚀

- Para arrancar hoy mismo con el **Paso 1 (Estructura y Base de Datos)**, solo tienen que ponerse de acuerdo en qué lenguaje prefieren para el backend:
- **Opción A — Node.js (JavaScript / TypeScript con Express o Fastify):** El estándar clásico de materias de Desarrollo Web, unifica todo el stack con el frontend.
 - **Opción B — Python (FastAPI):** Súper rápido de escribir, tipado moderno y muy cómodo para conectar scripts de IA.
- Cualquiera de las dos opciones es 100% válida y perfecta para cumplir con los requerimientos de la cátedra.